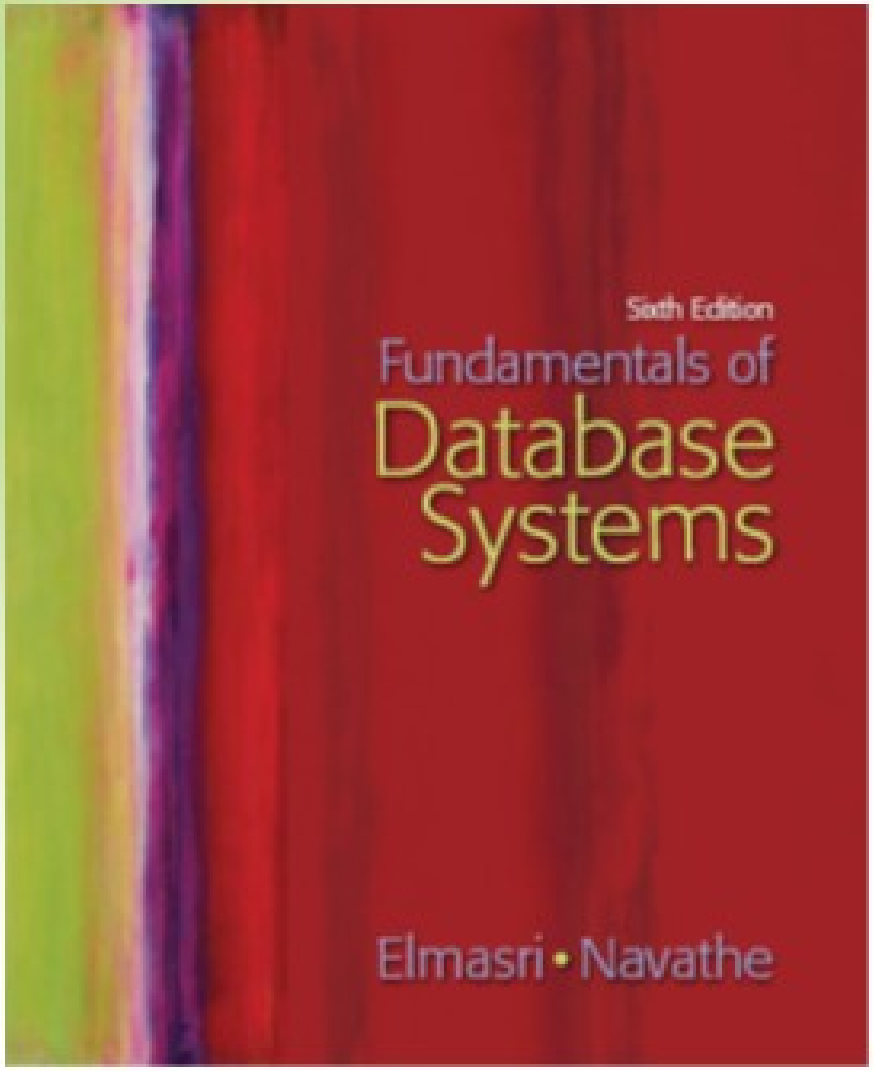
Chapter 17

Disk Storage, Basic File Structures, and Hashing



Addison-Wesley
is an imprint of
PEARSON

Copyright © 2011 Pearson Education, Inc. Publishing as Pearson Addison-Wesley

PEARSON
Addison Wesley

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

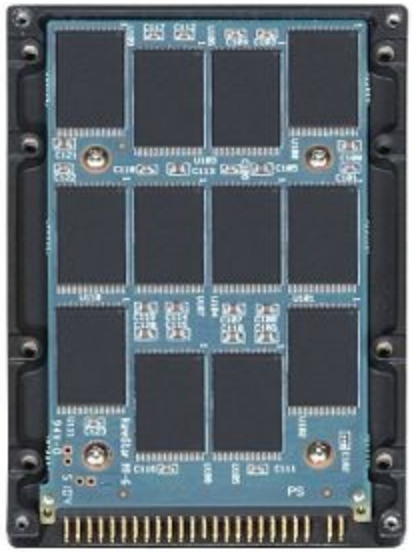
Chapter 17 - Outline

- Disk Storage Devices
- Files of Records
- Operations on Files
- Unordered Files
- Ordered Files
- Hashed Files
  - Dynamic and Extendible Hashing Techniques
- RAID Technology

Disk Storage Devices

- Electromechanical disks are the preferred secondary storage device for high storage capacity and low cost.
- Data stored as magnetized areas on magnetic disk surfaces.
- A **disk pack** contains several magnetic disks connected to a rotating spindle.
- Disks are divided into concentric circular **tracks** on each disk **surface**.
  - Track capacities vary typically from 4 to 50 Kbytes or more

Disk Storage Devices



- Open casing of 2.5-inch traditional hard disk drive (left) and solid-state drive (center).
  Picture from: http://en.wikipedia.org/wiki/Solid-state_drive

Disk Storage Devices (contd.)

- A track is divided into smaller **blocks** or **sectors**
  - because it usually contains a large amount of information
- The division of a track into **sectors** is hard-coded on the disk surface and cannot be changed.
  - One type of sector organization calls a portion of a track that subtends a fixed angle at the center as a sector.
- A track is divided into **blocks**.
  - The block size B is fixed for each system.
    - Typical block sizes range from B=512 bytes to B=4096 bytes.
  - Whole blocks are transferred between disk and main memory for processing.

Disk Storage Devices (contd.)

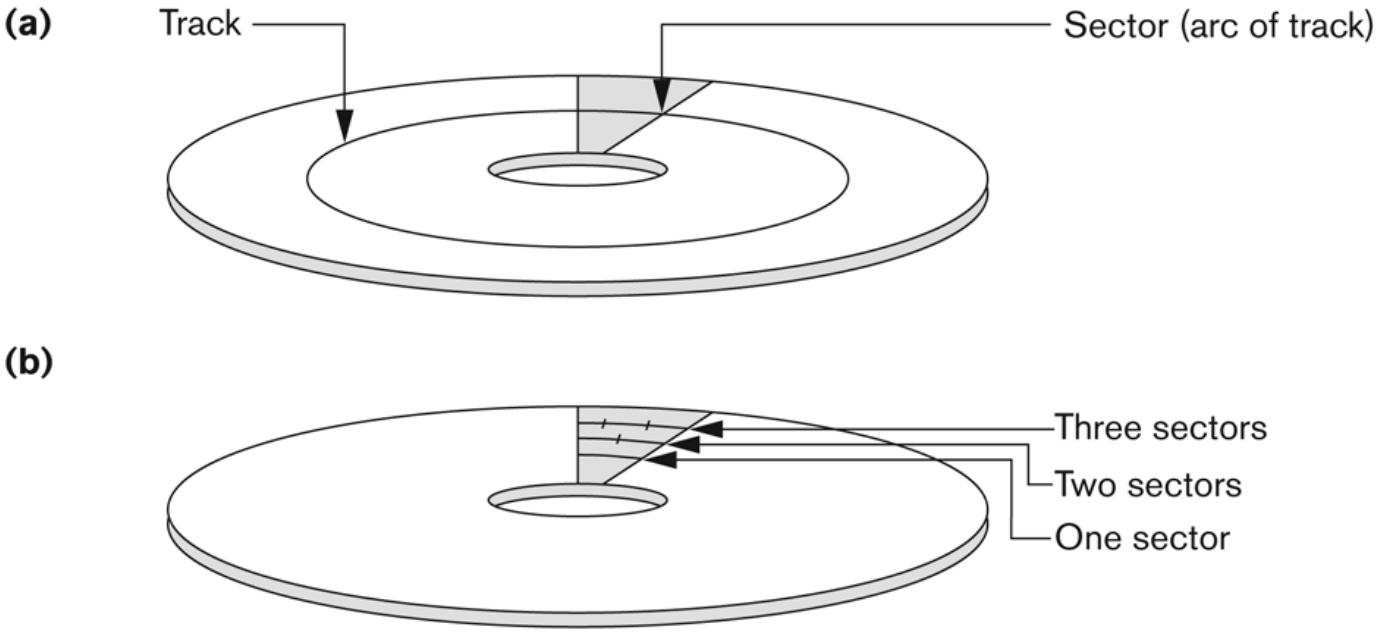


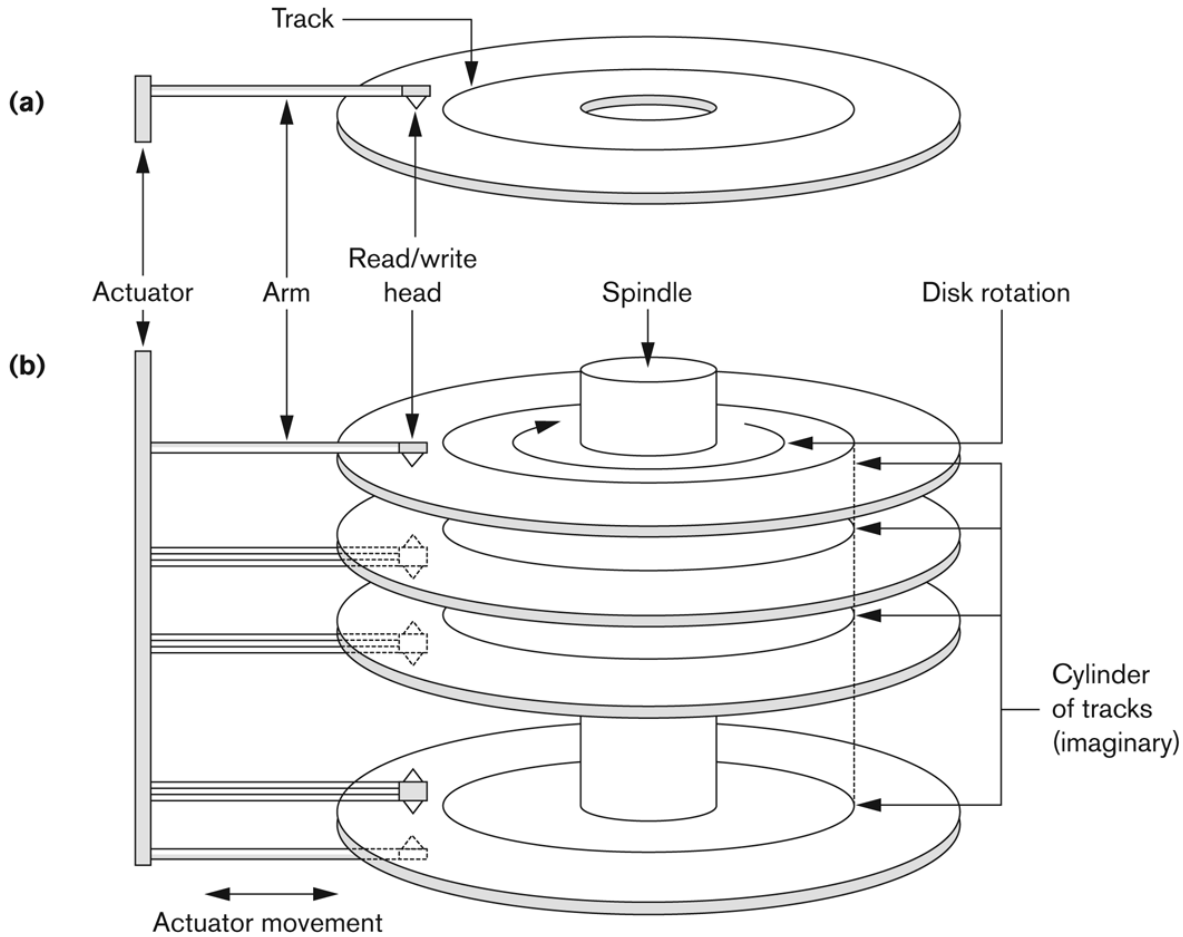
Figure 13.2
Different sector organizations on disk. (a) Sectors subtending a fixed angle. (b) Sectors maintaining a uniform recording density.

Disk Storage Devices (contd.)

- A **read-write head** moves to the track that contains the block to be transferred.
  - Disk rotation moves the block under the read-write head for reading or writing.
- A physical disk block (hardware) address consists of:
  - a cylinder number (imaginary collection of tracks of same radius from all recorded surfaces)
  - the track number or surface number (within the cylinder)
  - and block number (within track).
- Reading or writing a disk block is time consuming because of the seek time s and rotational delay (latency) **rd**.
- Double buffering can be used to speed up the transfer of contiguous disk blocks.

Disk Storage Devices (contd.)

Figure 13.1
(a) A single-sided disk with read/write hardware. (b) A disk pack with read/write hardware.



Typical Disk Parameters

Table 13.1
Specifications of Typical High-end Cheetah Disks from Seagate

Description	Cheetah 10K.6	Cheetah 10K.7
Model Number	ST3146807LC	ST3300007LW
Form Factor (width)	3.5 inch	3.5 inch
Form Factor (height)	1 inch	1 inch
Height	25.4 mm	25.4 mm
Width	101.6 mm	101.6 mm
Length	146.05 mm	146.05 mm
Weight	0.73 Kg	0.726 kg
Capacity/Interface		
Formatted Capacity	146.8 Gbytes	300 Gbytes
Interface Type	80-pin	68-pin
Configuration		
Number of disks (physical)	4	4
Number of heads (physical)	8	8
Number of Cylinders	49,854	90,774
Total Tracks		726,192
Bytes per Sector	512	512
Areal Density	36,000 Mb/sq.inch	
Track Density	64,000 Tracks/inch	105,000 Tracks/inch
Recording Density	570,000 bits/inch	658,000 bits/inch
Bytes/Track (avg)		556
Performance		
Transfer Rates		
Internal Transfer Rate (min)	475 Mb/sec	472 Mb/sec
Internal Transfer Rate (max)	840 Mb/sec	944 Mb/sec
Formated Int. Transfer Rate (min)	43 MB/sec	59 MB/sec
Formated Int. Transfer Rate (max)	78 MB/sec	118 MB/sec
External I/O Transfer Rate (max)	320 MB/sec	320 MB/sec
Average Formatted Transfer Rate	59.9 MB/sec	59.5 MB/sec
Seek Times		
Avg. Seek Time (Read)	4.7 ms (typical)	4.7 ms (typical)
Avg. Seek Time (Write)	5.2 ms (typical)	5.3 ms (typical)
Track-to-track, Seek, Read	0.3 ms (typical)	0.2 ms (typical)
Track-to-track, Seek, Write	0.5 ms (typical)	0.5 ms (typical)
Full Disc Seek, Read		9.5 ms (typical)
Full Disc Seek, Write		10.3 ms (typical)
Average Latency	2.99 ms	3 msec
Other		
Default Buffer (cache) size	8,000 KB	8,192 KB
Spindle Speed	10000 RPM	10000 RPM
Power-on to Ready Time		25 sec

(Courtesy of Seagate Technology)

Records

- **Fixed** and **variable** length records
- Records contain **fields** which have values of a particular type
  - E.g., *amount, date, time, age*
- Fields themselves may be **fixed** length or **variable** length
- Variable length fields can be mixed into one record:
  - Separator characters or length fields are needed so that the record can be “parsed.”

Blocking

- **Blocking**:
  - Refers to storing a number of records in one block on the disk.
- Blocking factor (**bfr**) refers to the number of records per block.
- There may be empty space in a block if an integral number of records do not fit in one block.
- **Spanned Records**:
  - Refers to records that exceed the size of one or more blocks and hence span a number of blocks.

Files of Records

- A **file** is a *sequence* of records, where each record is a collection of data values (or data items).
- A **file descriptor** (or **file header**) includes information that describes the file, such as the *field names* and their *data types*, and the addresses of the file blocks on disk.
- Records are stored on disk blocks.
- The **blocking factor bfr** for a file is the (average) number of file records stored in a disk block.
- A file can have **fixed-length** records or **variable-length** records.

Files of Records

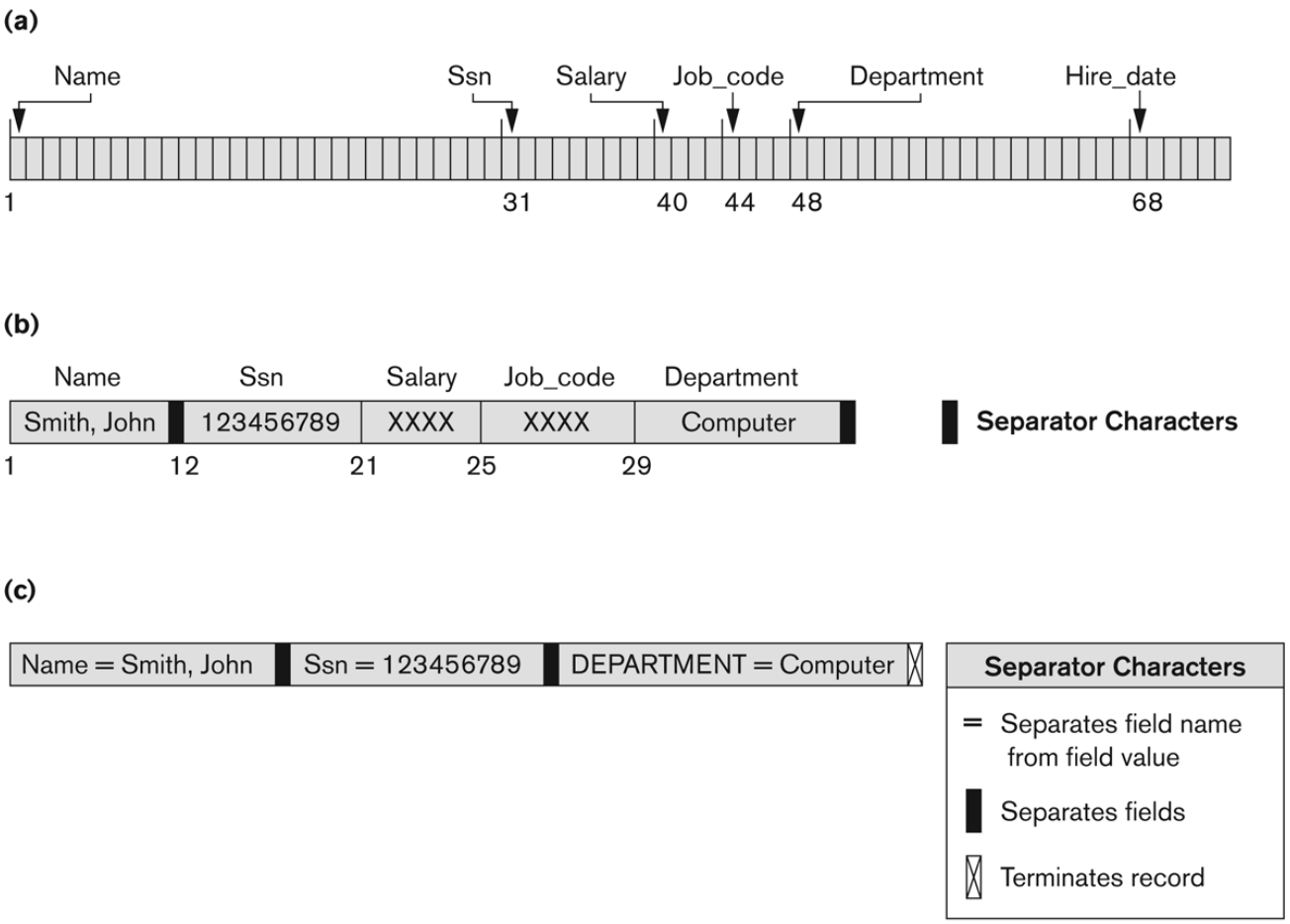


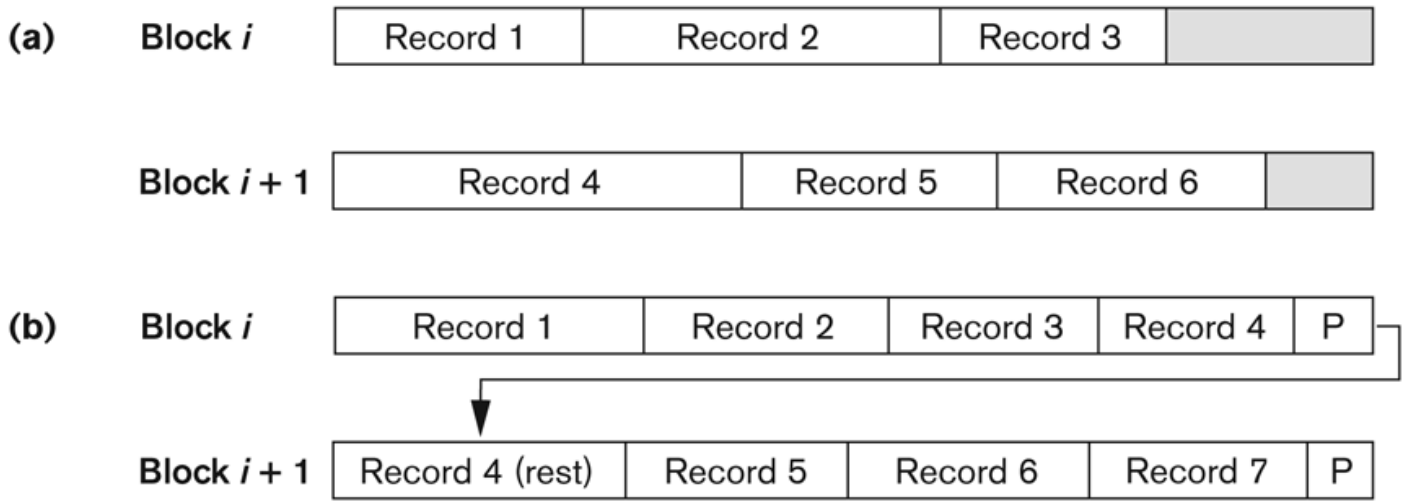
Figure 13.5
Three record storage formats. (a) A fixed-length record with six fields and size of 71 bytes. (b) A record with two variable-length fields and three fixed-length fields. (c) A variable-field record with three types of separator characters.

Files of Records (contd.)

- File records can be **unspanned** or **spanned**
  - **Unspanned**: no record can span two blocks
  - **Spanned**: a record can be stored in more than one block
- The physical disk blocks that are allocated to hold the records of a file can be *contiguous, linked, or indexed*.
- In a file of fixed-length records, all records have the same format. Usually, unspanned blocking is used with such files.
- Files of variable-length records require additional information to be stored in each record, such as **separator characters** and **field types**.
  - Usually spanned blocking is used with such files.

Files of Records (contd.)

Figure 13.6
Types of record organization. (a) Unspanned. (b) Spanned.



Operation on Files

- Typical file operations include:
  - **OPEN**: Readies the file for access, and associates a pointer that will refer to a *current* file record at each point in time.
  - **FIND**: Searches for the first file record that satisfies a certain condition, and makes it the current file record.
  - **FINDNEXT**: Searches for the next file record (from the current record) that satisfies a certain condition, and makes it the current file record.
  - **READ**: Reads the current file record into a program variable.
  - **INSERT**: Inserts a new record into the file & makes it the current file record.
  - **DELETE**: Removes the current file record from the file, usually by marking the record to indicate that it is no longer valid.
  - **MODIFY**: Changes the values of some fields of the current file record.
  - **CLOSE**: Terminates access to the file.
  - **REORGANIZE**: Reorganizes the file records.
    - For example, the records marked deleted are physically removed from the file or a new organization of the file records is created.
  - **READ_ORDERED**: Read the file blocks in order of a specific field of the file.

Unordered Files

- Also called a **heap** or a **pile** file.
- New records are inserted at the end of the file.
- A **linear search** through the file records is necessary to search for a record.
  - This requires reading and searching half the file blocks on the average, and is hence quite expensive.
- Record insertion is quite efficient.
- Reading the records in order of a particular field requires sorting the file records.

Ordered Files

- Also called a **sequential** file.
- File records are kept sorted by the values of an *ordering field*.
- Insertion is expensive: records must be inserted in the correct order.
  - It is common to keep a separate unordered *overflow* (or *transaction*) file for new records to improve insertion efficiency; this is periodically merged with the main ordered file.
- A **binary search** can be used to search for a record on its *ordering field* value.
  - This requires reading and searching $\log_2$ of the file blocks on the average, an improvement over linear search.
- Reading the records in order of the ordering field is quite efficient.

Ordered Files (contd.)

Try inserting
Adama, Williams

	Name	Ssn	Birth_date	Job	Salary	Sex
Block 1	Aaron, Ed					
	Abbott, Diane					
	⋮					
	Acosta, Marc					
Block 2	Adams, John					
	Adams, Robin					
	⋮					
	Akers, Jan					
Block 3	Alexander, Ed					
	Alfred, Bob					
	⋮					
	Allen, Sam					
Block 4	Allen, Troy					
	Anders, Keith					
	⋮					
	Anderson, Rob					
Block 5	Anderson, Zach					
	Angeli, Joe					
	⋮					
	Archer, Sue					
Block 6	Arnold, Mack					
	Arnold, Steven					
	⋮					
	Atkins, Timothy					
	⋮					
Block *n*−1	Wong, James					
	Wood, Donald					
	⋮					
	Woods, Manny					
Block *n*	Wright, Pam					
	Wyatt, Charles					
	⋮					
	Zimmer, Byron					

Figure 13.7
Some blocks of an ordered (sequential) file of EMPLOYEE records with Name as the ordering key field.

Average Access Times

- The following table shows the average access time to access a specific record for a given type of file

TABLE 13.2 AVERAGE ACCESS TIMES FOR BASIC FILE ORGANIZATIONS

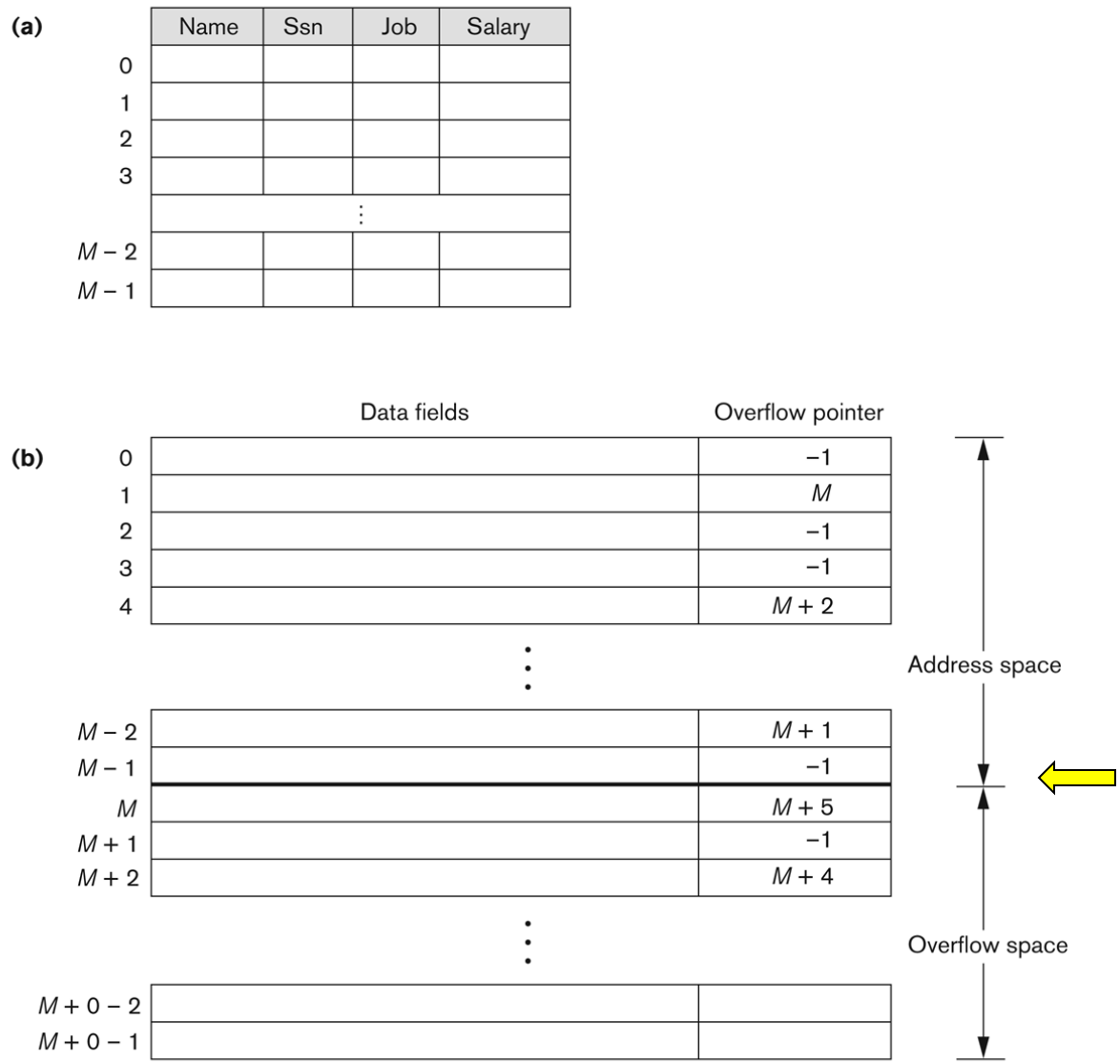
TYPE OF ORGANIZATION	ACCESS/SEARCH METHOD	AVERAGE TIME TO ACCESS A SPECIFIC RECORD
Heap (Unordered)	Sequential scan (Linear Search)	$b/2$
Ordered	Sequential scan	$b/2$
Ordered	Binary Search	$\log_2 b$

Hashed Collections

- Small collections that can fit in memory could be stored using an internal hashing organization.
- A simple hashing function is *(location)*:

  h(key) = Key *mod* M

Figure 13.8
Internal hashing data structures. (a) Array of *M* positions for use in internal hashing. (b) Collision resolution by chaining records.



- null pointer = –1
- overflow pointer refers to position of next record in linked list

Hashed Files

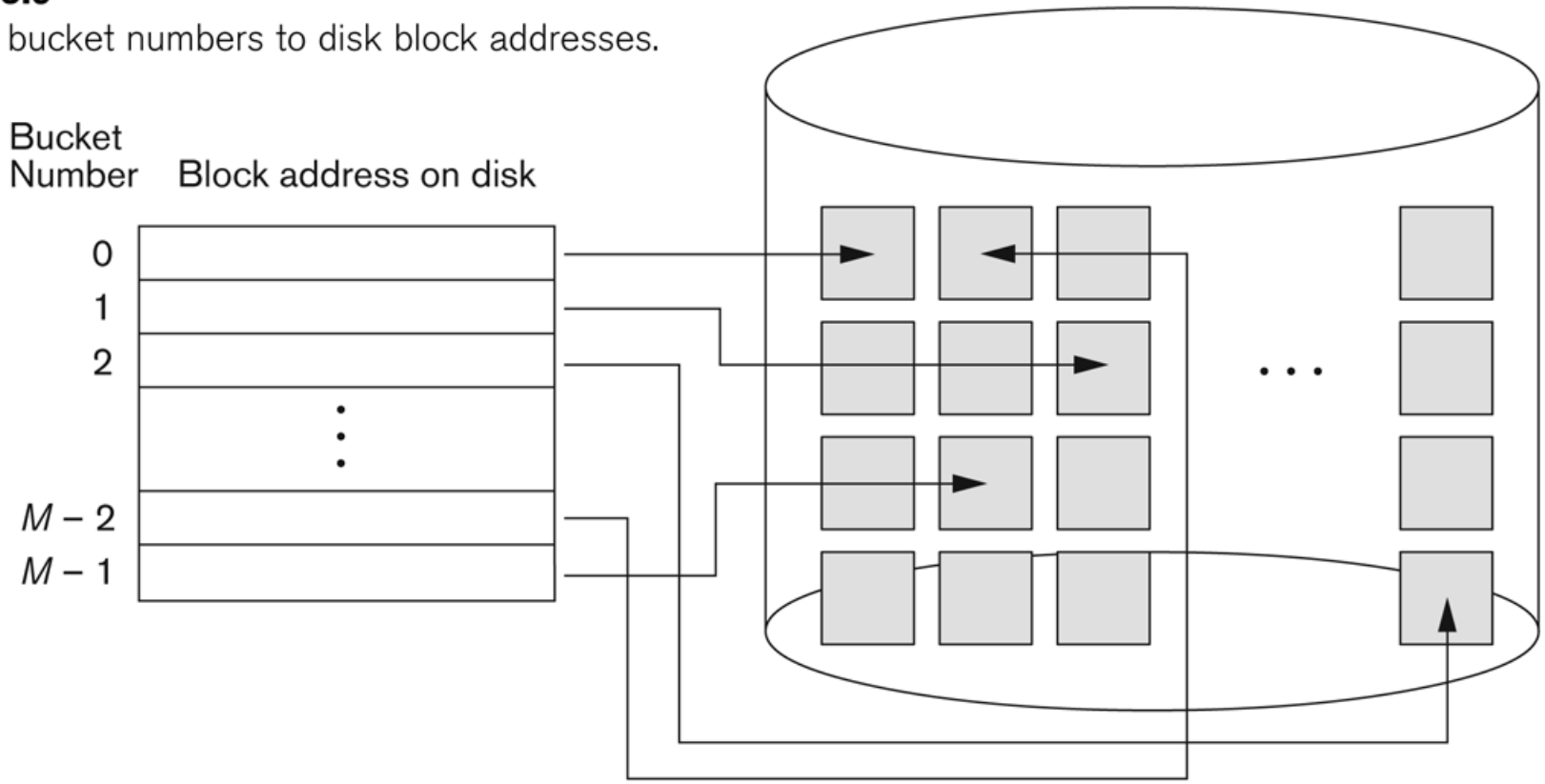
- Hashing for disk files is called **External Hashing**
- The file blocks are divided into M equal-sized **buckets**, numbered $bucket_0$, $bucket_1$, ..., $bucket_{M-1}$.
  - Typically, a bucket corresponds to one (or a fixed number of) disk block.
- One of the file fields is designated to be the **hash key** of the file.
- The record with hash key value K is stored in bucket i, where **i=h(K)**, and h is the **hashing function**.
- Search is very efficient on the hash key.
- Collisions occur when a new record hashes to a bucket that is already full.
  - An overflow file is kept for storing such records.
  - Overflow records that hash to each bucket can be linked together.

Hashed Files (contd.)

- There are numerous methods for *collision resolution*, including the following:
  - **Open addressing**:
    Proceeding from the occupied position specified by the hash address, the program checks the subsequent positions in order until an unused (empty) position is found.
  - **Chaining**:
    For this method, various overflow locations are kept, usually by extending the array with a number of overflow positions. In addition, a pointer field is added to each record location. A collision is resolved by placing the new record in an unused overflow location and setting the pointer of the occupied hash address location to the address of that overflow location.
  - **Multiple hashing**:
    The program applies a second hash function if the first results in a collision. If another collision results, the program uses open addressing or applies a third hash function and then uses open addressing if necessary.

Hashed Files (contd.)

Figure 13.9
Matching bucket numbers to disk block addresses.



Hashed Files (contd.)

- To reduce overflow records, a hash file is typically kept 70-80% full.
- The hash function h should distribute the records uniformly among the buckets
  - Otherwise, search time will be increased because many overflow records will exist.
- Main disadvantages of static external hashing:
  - Fixed number of buckets M is a problem if the number of records in the file grows or shrinks.
  - Ordered access on the hash key is quite inefficient (requires sorting the records).

Hashed Files - Overflow handling

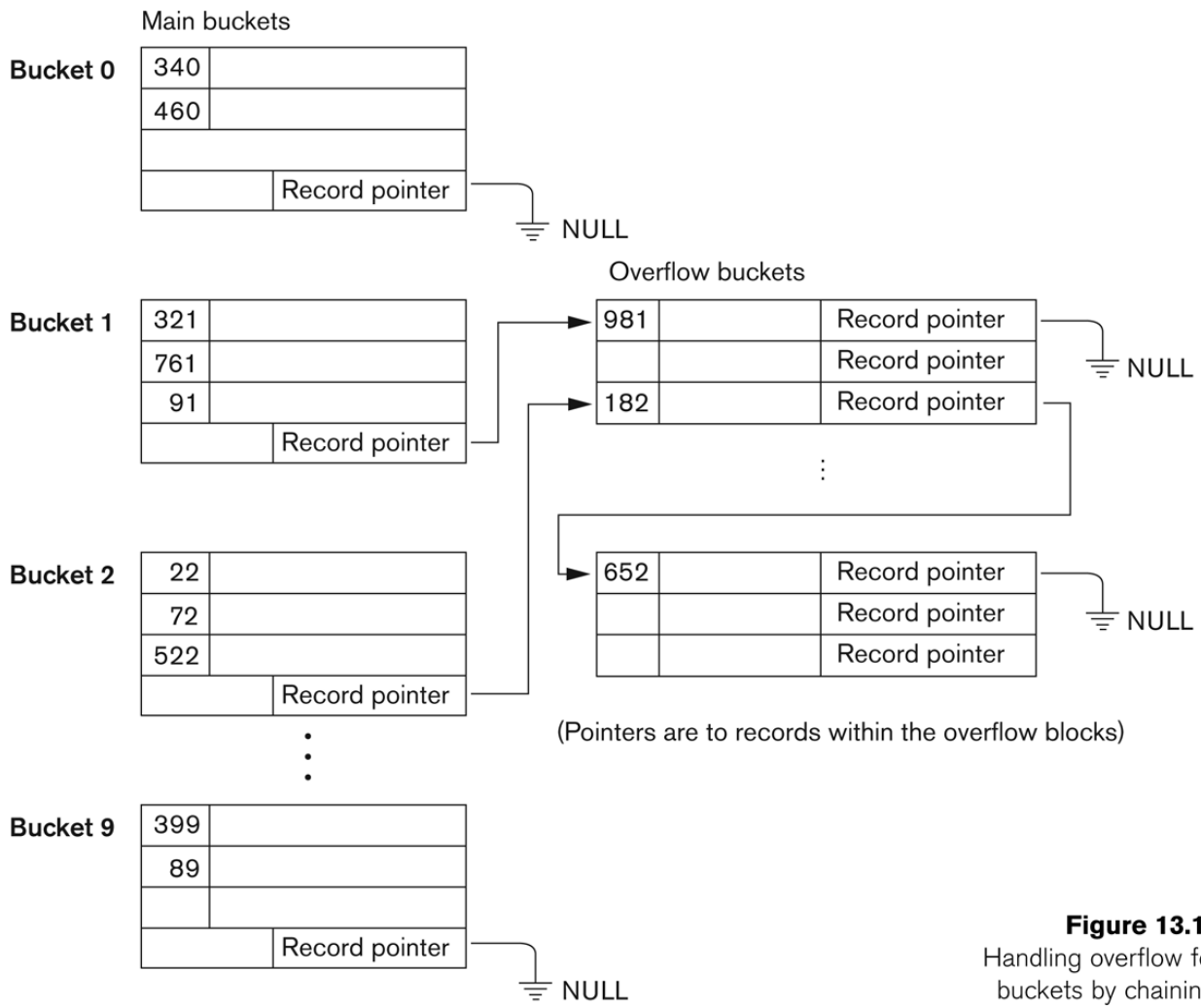


Figure 13.10
Handling overflow for buckets by chaining.

Dynamic And Extendible Hashed Files

- **Dynamic and Extendible Hashing Techniques**
  - Hashing techniques are adapted to allow the dynamic growth and shrinking of the number of file records.
  - These techniques include the following:
    - **dynamic hashing,**
    - **extendible hashing**, and
    - **linear hashing.**

- Both **dynamic** and **extendible** hashing use the **binary representation** of the hash value h(K) in order to access a **directory**.
  - In dynamic hashing the directory is a *binary tree*.
  - In extendible hashing the directory is an array of size 2^d where d is called the *global depth.*

Dynamic And Extendible Hashing (contd.)

- The directories can be stored on disk, and they expand or shrink dynamically.
  - Directory entries point to the disk blocks that contain the stored records.
- An insertion in a disk block that is full causes the block to split into two blocks and the records are redistributed among the two blocks.
  - The directory is updated appropriately.
- Dynamic and extendible hashing do not require an overflow area.
- Linear hashing does require an overflow area but does not use a directory.
  - Blocks are split in *linear order* as the file expands (one *linear* bucket at the time).

Extendible Hashing

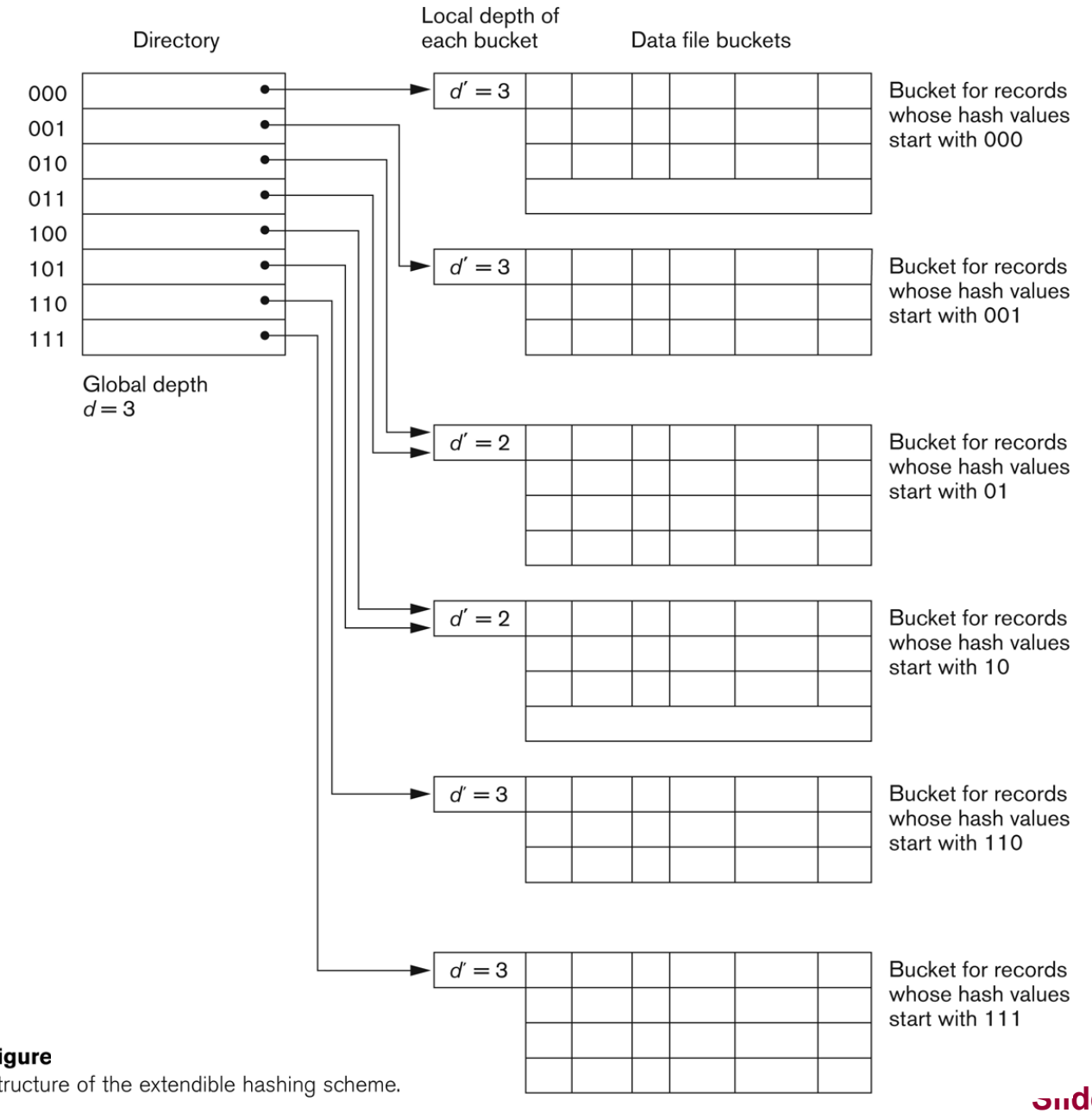
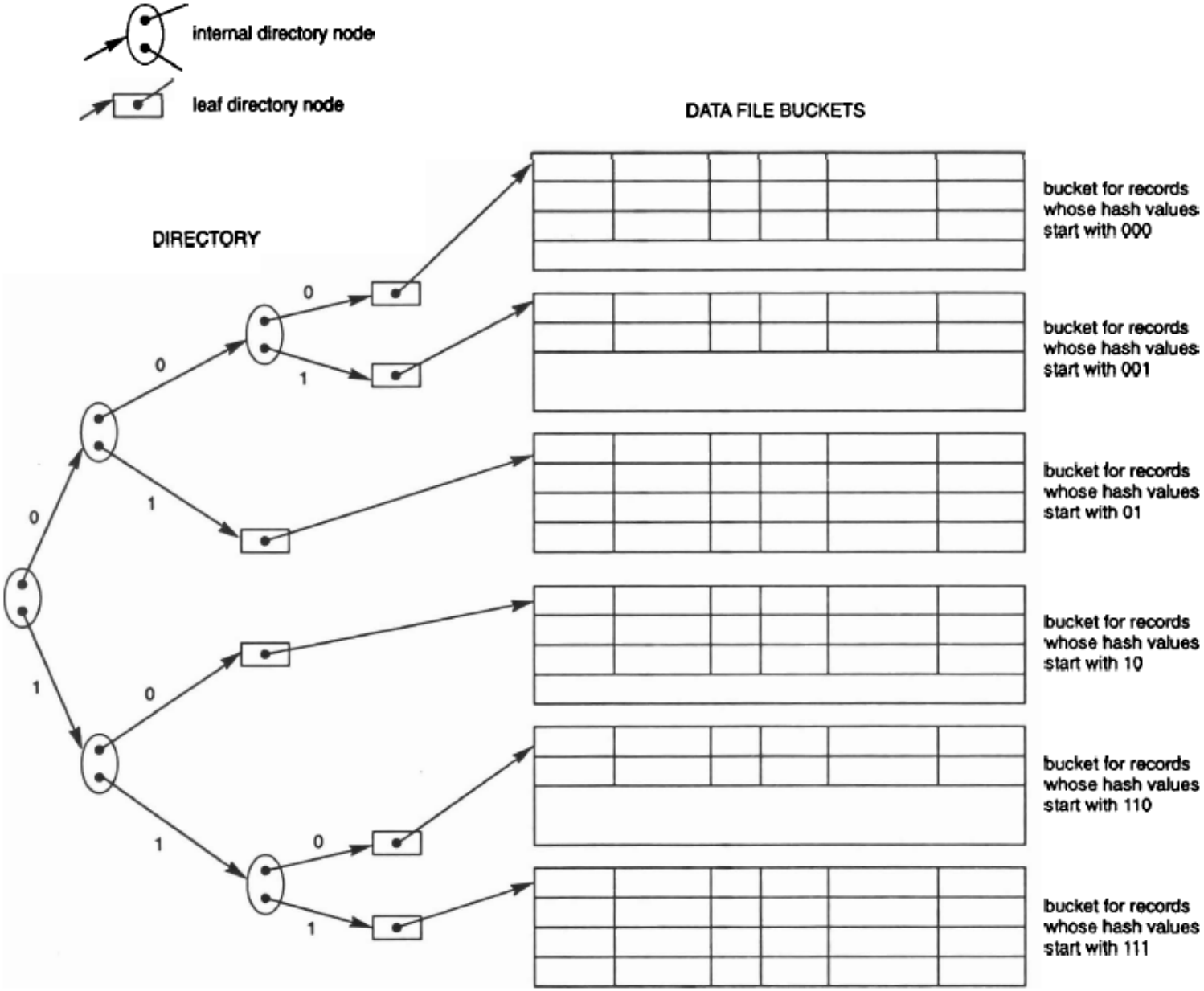


Figure
Structure of the extendible hashing scheme.

Dynamic Hashing



Extendible & Dynamic Hashing

Example

Organize the following file using the previous schemas

Key	binary	SampleData
01	0001	data1
07	0111	data2
03	0011	data3
08	1000	data4
12	1100	data5
14	1110	data6
11	1011	data7
02	0010	data8
10	1010	data9
13	1101	data10
04	0100	data11
09	1001	data12

Each record consists of a **key** and **data**.

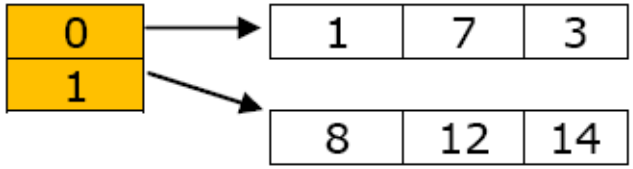
The binary representation of the key (using 4-bits) is provided for your convenience

Dynamic Hashing

Inserting Key Values – Dynamic Hashing

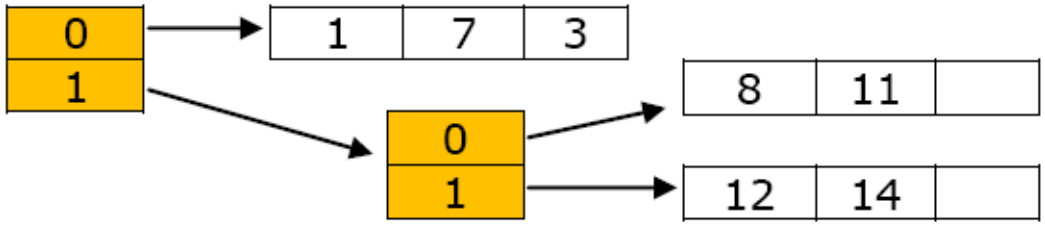
{ 1, 7, 3, 8, 12, 14, 11, 2, 10, 13, 4, 9 }

After inserting 1, 7, 3, 8, 12, 14



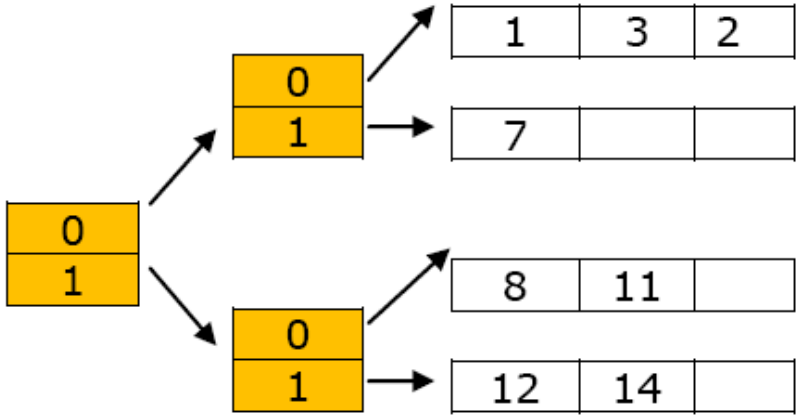

```
01 0001 data1
07 0111 data2
03 0011 data3
08 1000 data4
12 1100 data5
14 1110 data6
11 1011 data7
02 0010 data8
10 1010 data9
13 1101 data10
04 0100 data11
09 1001 data12
```

Inserting 11 produces overflow on bucket$_1$



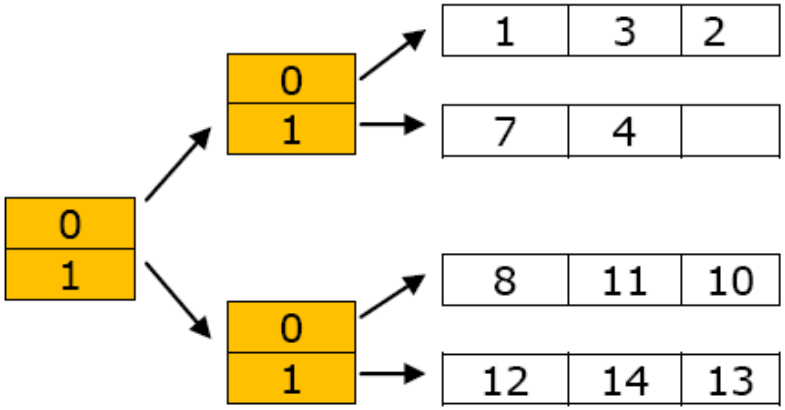
Dynamic Hashing

Inserting 2 produces overflow in bucket$_0$




```
01 0001 data1
07 0111 data2
03 0011 data3
08 1000 data4
12 1100 data5
14 1110 data6
11 1011 data7
02 0010 data8
10 1010 data9
13 1101 data10
04 0100 data11
09 1001 data12
```

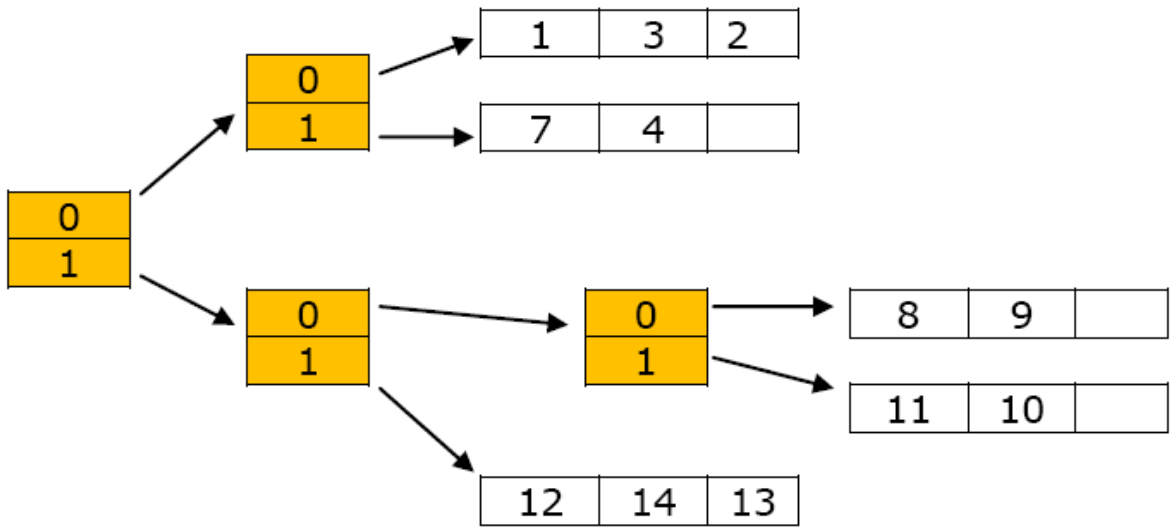
Inserting 10, 13, and 4



Dynamic Hashing

```
01 0001 data1
07 0111 data2
03 0011 data3
08 1000 data4
12 1100 data5
14 1110 data6
11 1011 data7
02 0010 data8
10 1010 data9
13 1101 data10
04 0100 data11
09 1001 data12
```

Inserting 9 produces overflow of Bucket$_{1,0}$

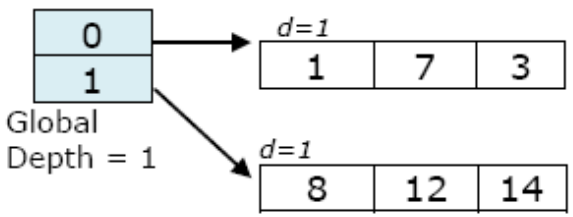


Extendible Hashing

Inserting Key Values – Extendible Hashing

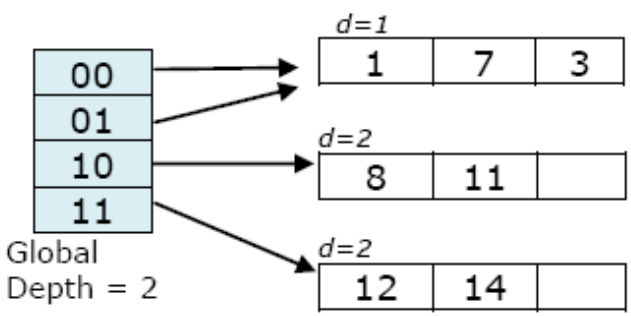
{ 1, 7, 3, 8, 12, 14, 11, 2, 10, 13, 4, 9 }

After inserting 1, 7, 3, 8, 12, 14



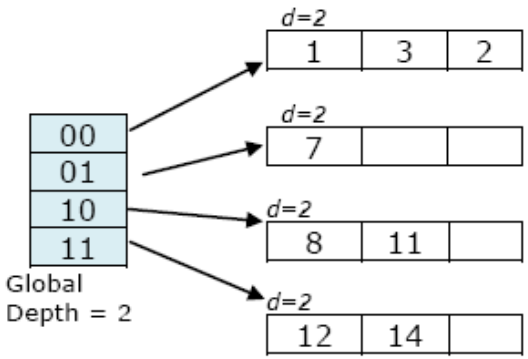

```
01 0001 data1
07 0111 data2
03 0011 data3
08 1000 data4
12 1100 data5
14 1110 data6
11 1011 data7
02 0010 data8
10 1010 data9
13 1101 data10
04 0100 data11
09 1001 data12
```

Inserting 11 produces overflow on bucket$_1$



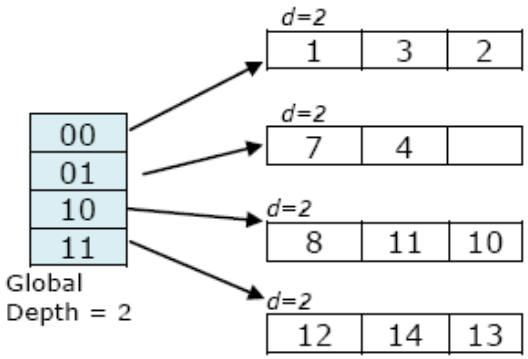
Extendible Hashing

Inserting 2 produces overflow of Bucket $_0$



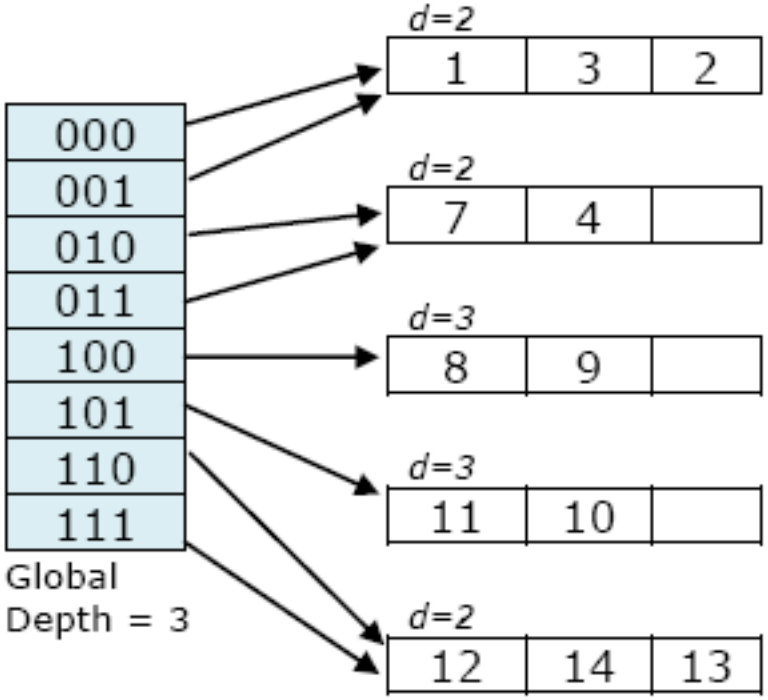

```
01 0001 data1
07 0111 data2
03 0011 data3
08 1000 data4
12 1100 data5
14 1110 data6
11 1011 data7
02 0010 data8
10 1010 data9
13 1101 data10
04 0100 data11
09 1001 data12
```

Continue inserting 10, 13, and 4



Extendible Hashing

Inserting 9 produces overflow of Bucket$_{10}$




```
01 0001 data1
07 0111 data2
03 0011 data3
08 1000 data4
12 1100 data5
14 1110 data6
11 1011 data7
02 0010 data8
10 1010 data9
13 1101 data10
04 0100 data11
09 1001 data12
```

Linear Hashing

How does it work?

- Consider a hash table consisting of **N** buckets with addresses *0, 1, . . . , N - 1.*
- Linear hashing increases the address space gradually by splitting the buckets in a predetermined order: first bucket 0, then bucket 1, and so on, up to and including bucket N - 1. This current position is retained by the *split variable* ***s***.

- Splitting a bucket involves moving approximately half of the records from the **s** bucket to a new bucket at the end of the table.

- Suppose the placement of data is done using the hashing function **$h_1(k) = k \bmod N$**. The corresponding bucket location is determined as follows:

$$\text{if } (h_1(k) < s) \text{ then } h_2(k) \text{ else } h_1(k);$$

Linear Hashing

How does it work? – Dealing with OVERFLOW

Whenever there is an **overflow** situation follow the next steps,

1. the new data item is attached to the overflowing bucket using chain-pointing.

2. a new node is added to the end of the bucket area (N, N+1, …)

3. the current splitting bucket ($Bucket_s$) is rehashed using the function **$h_2(k) = k \bmod 2*N$**.

4. The split variable ***s*** is incremented to the next position.

5. If *s* becomes N, it is reset to zero and the $h_1(k)$ is replaced by $h_2(k)$. At this point the bucket space has duplicated.

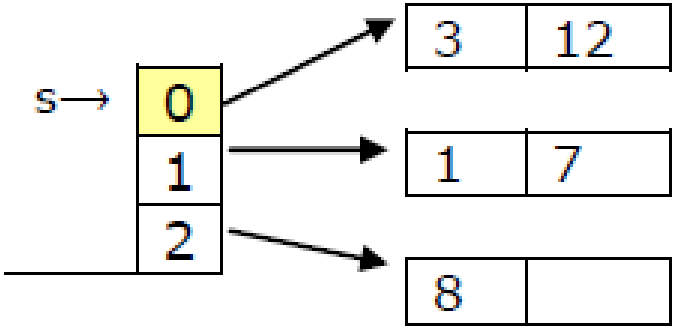
Linear Hashing

Example

Consider the key set given below. Hashing parameter M=3; therefore use the functions $h_1(K) = k \; mod \; 3$ and $h_2(K) = k \bmod 6$. The bucket size is 2.

Keys = { **1, 7, 3, 8, 12**, 4, 11, 2, 10, 13, 14, 9 }

Inserting **1, 7, 3, 8, 12**



Mapping keys

$h_1(1) = 1 \; mod \; 3 = 1$
$h_1(7) = 7 \; mod \; 3 = 1$
$h_1(3) = 3 \; mod \; 3 = 0$
$h_1(8) = 8 \; mod \; 3 = 2$
$h_1(12) = 12 \; mod \; 3 = 0$

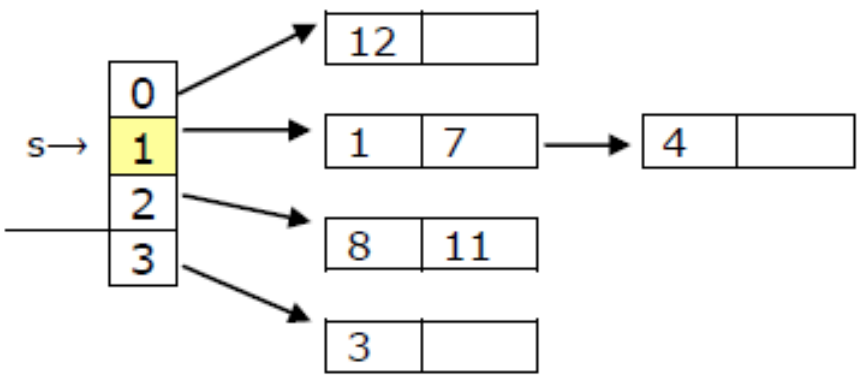
Linear Hashing

Example *continuation* Keys = { 1, 7, 3, 8, 12, **4**, 11, 2, 10, 13, 14, 9 }

Inserting **4** produces overflow of $Bucket_1$ ($h_1(4) = 4 \; mod \; 3 = 1$).

On overflow we do

(a) use chain-linking to add 4 to $Bucket_1$.
(b) split $Bucket_s$ (s=0) and remap its keys using $h_2(k) = k \; mod \; 6$
(c) advance split pointer s (s=1)



Mapping / Remapping keys

$h_1(4) = 4 \; mod \; 3 = 1$ *Overflow !!!*

Chain-Link 4 to $Bucket_1$
Add entry 3 to directory.
Remapping $Bucket_0$ (s = 0)
$h_2(3) = 3 \; mod \; 6 = 3$
$h_2(12) = 12 \; mod \; 6 = 0$

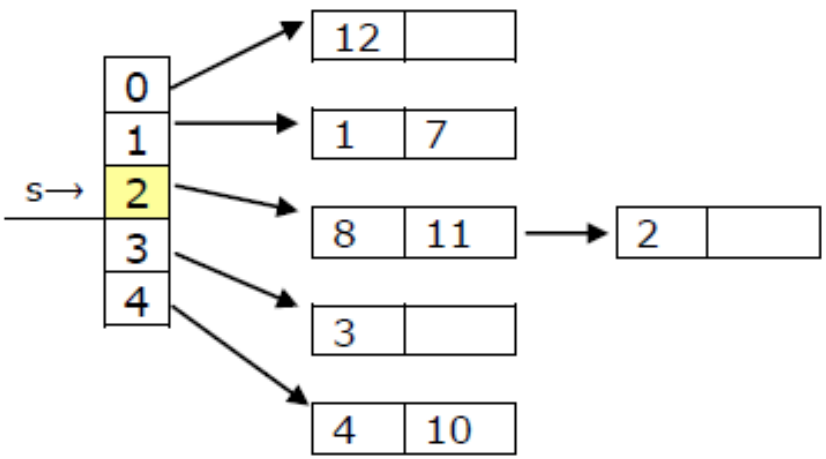
Advance s (s=1)

Continue with key 11
$h_1(11) = 11 \; mod \; 3 = 2$

Linear Hashing

Example *continuation* Keys = { 1, 7, 3, 8, 12, 4, 11, **2**, **10**, 13, 14, 9 }

Continue inserting **2**; this produces overflow of $Bucket_2$.



Mapping keys

$h_1(2) = 2 \; mod \; 3 = 2$ *Overflow !!!*

OVF Actions

Chain-Link 2 to $Bucket_2$

Add entry 4 to directory

Remapping $Bucket_s$ (s=1)

$h_2(1) = 1 \; mod \; 6 = 1$

$h_2(7) = 7 \; mod \; 6 = 1$

$h_2(4) = 7 \; mod \; 6 = 4$

Advance s (s=2)

Continue with key 10

$h_1(10) = 10 \; mod \; 3 = 1$

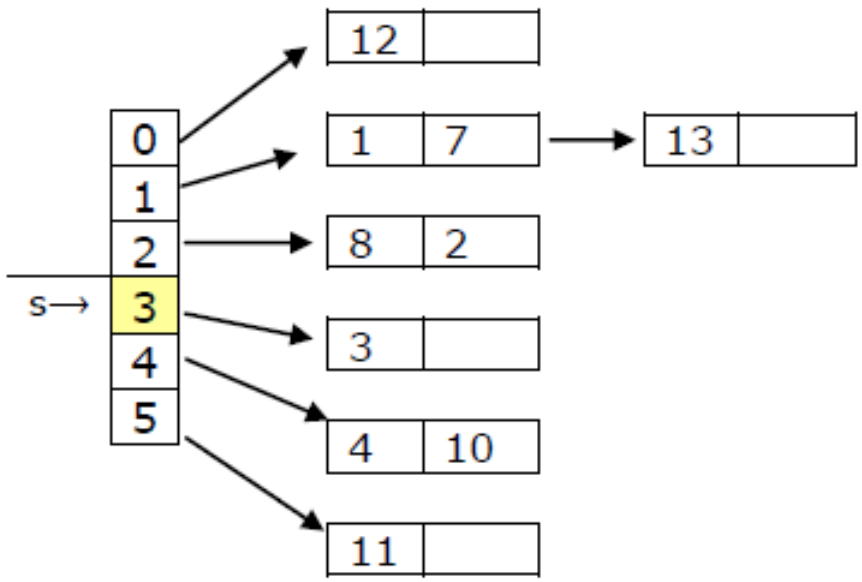
Observe $h_1(10) = 1 < s$

therefore must use $h_2(10) = 10 \; mod \; 6 = 4$

Linear Hashing

Example *continuation* Keys = { 1, 7, 3, 8, 12, 4, 11, 2, 10, **13**, 14, 9 }

Continue inserting **13**; produces overflow of $Bucket_1$



Mapping keys

$h_1(13) = 13 \; mod \; 3 = 1$ *Overflow !!!*

OVF Actions

Chain-Link 13 to $Bucket_1$

Add entry 5 to directory

Remapping $Bucket_s$ (s=2)

$h_2(8) = 8 \; mod \; 6 = 2$

$h_2(11) = 11 \; mod \; 6 = 5$

$h_2(2) = 2 \; mod \; 6 = 2$

Advance s (s=3), it becomes N

After resetting

N= 6, s = 0, and mapping functions

$h_1(k)$ and $h_2(k)$ change to

$h_1(k) = k \; mod \; 6$

$h_2(k) = k \; mod \; 12$

Linear Hashing

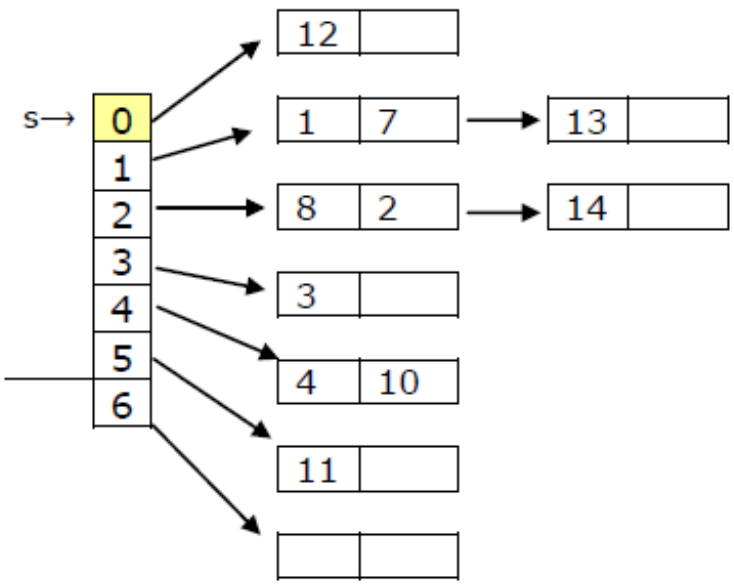
Example *continuation* Keys = { 1, 7, 3, 8, 12, 4, 11, 2, 10, 13, **14**, 9 }

NOTE

After inserting **13** the splitting variable **s** reaches the value N=3. At this point the 'old' mapping space has duplicated in size. Therefore after resetting new **N=6**, **s=0** and the the original hashing function $h_1(k)= k \; mod \; 3$ must be changed to

$h_1(k)= k \; mod \; 6$, and **$h_2(k)= k \; mod \; 12$.**

Continue inserting **14**. This produces overflow of $Bucket_2$.



Mapping keys

$h_1(14) = 14 \; mod \; 6 = 2$ *Overflow !!!*

OVF Actions

Chain-Link 14 to $Bucket_2$

Add new key 6 to directory.

Remapping $Bucket_s$ (s=0)

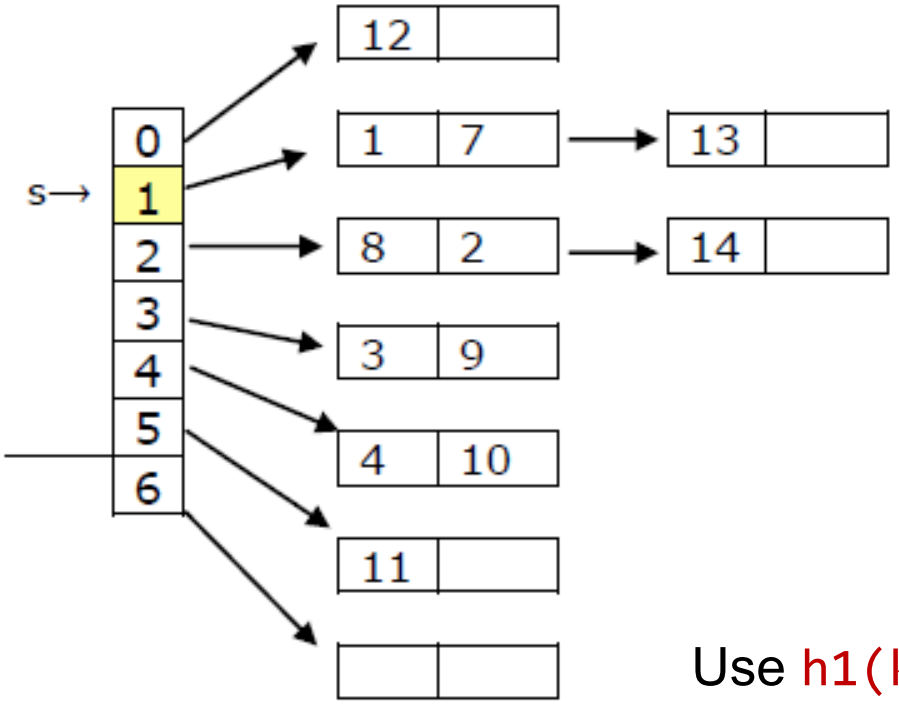
$h_2(12) = 12 \; mod \; 12 = 0$

Advance s (s=1)

Linear Hashing

Example *continuation* Keys = { 1, 7, 3, 8, 12, 4, 11, 2, 10, 13, 14, **9** }

Continue inserting **9** (no conflits) . This terminates the insertion process.



Mapping keys

$h_1(9) = 9 \; mod \; 6 = 3$

Remember control variables & functions:

N = 6
s = 1

h1(k) = k *mod* 6

h2(k) = k *mod* 12

Use `h1(k)= k mod 6`
to locate proper bucket

Linear Hashing

Example *continuation* Keys = { 1, 7, 3, 8, 12, 4, 11, 2, 10, 13, 14, 9 }

Note.
The final search function **Bucket** $\leftarrow$ **h_1(k)** is computed as:

```
h1(k) = k mod 6;
if (h1(k) < s) {
    h1(k) = k mod 12;
}
```

Extendible, Dynamic & Linear Hashing

Your turn…

Try to organize the following file using the previous schemas

```
20 00010100     data1
40 00101000     data2
10 00001010     data3
30 00011110     data4
15 00001111     data5
35 00100011     data6
07 00000111     data7
26 00011010     data8
18 00010010     data9
22 00010110     data10
05 00000101     data11
42 00101010     data12
13 00001101     data13
46 00101110     data14
27 00011011     data15
08 00001000     data16
32 00100000     data17
38 00100110     data18
24 00011000     data19
45 00101101     data20
25 00011001     data21
```

Each record consists of a **key** and **data**.

The binary representation of the key is provided for your convenience

Parallelizing Disk Access using RAID Technology.

- Secondary storage technology must take steps to keep up in performance and reliability with processor technology.
- A major advance in secondary storage technology is represented by the development of **RAID**, which originally stood for **Redundant Arrays of Inexpensive Disks**.
- The main goal of RAID is to even out the widely different rates of performance improvement of disks against those in memory and microprocessors.

RAID Technology (contd.)

- A natural solution is a large array of small independent disks acting as a single higher-performance logical disk.
- A concept called **data striping** is used, which utilizes parallelism to improve disk performance.
- Data striping distributes data transparently over multiple disks to make them appear as a single large, fast disk.

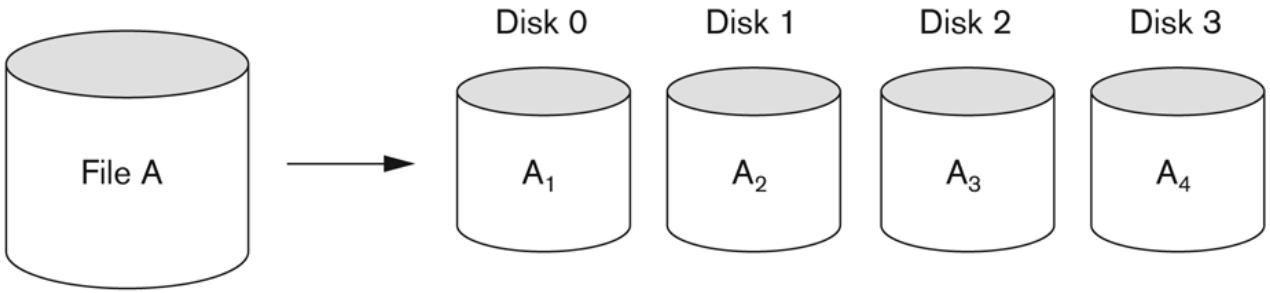


Figure 13.12
Data striping. File A is striped across four disks.

RAID Technology (contd.)

- For a tutorial on RAID technology see
- http://www.acnc.com/04_01_00.html

RAID Technology (contd.)

- Different raid organizations were defined based on different combinations of the two factors of **granularity of data interleaving** (striping) and **pattern** used to compute redundant information.
  - Raid level 0 has no redundant data and hence has the best write performance at the risk of data loss
  - Raid level 1 uses mirrored disks.
  - Raid level 2 uses memory-style redundancy by using Hamming codes, which contain parity bits for distinct overlapping subsets of components. Level 2 includes both error detection and correction.
  - Raid level 3 uses a single parity disk relying on the disk controller to figure out which disk has failed.
  - Raid Levels 4 and 5 use block-level data striping, with level 5 distributing data and parity information across all disks.
  - Raid level 6 applies the so-called P + Q redundancy scheme using Reed-Soloman codes to protect against up to two disk failures by using just two redundant disks.

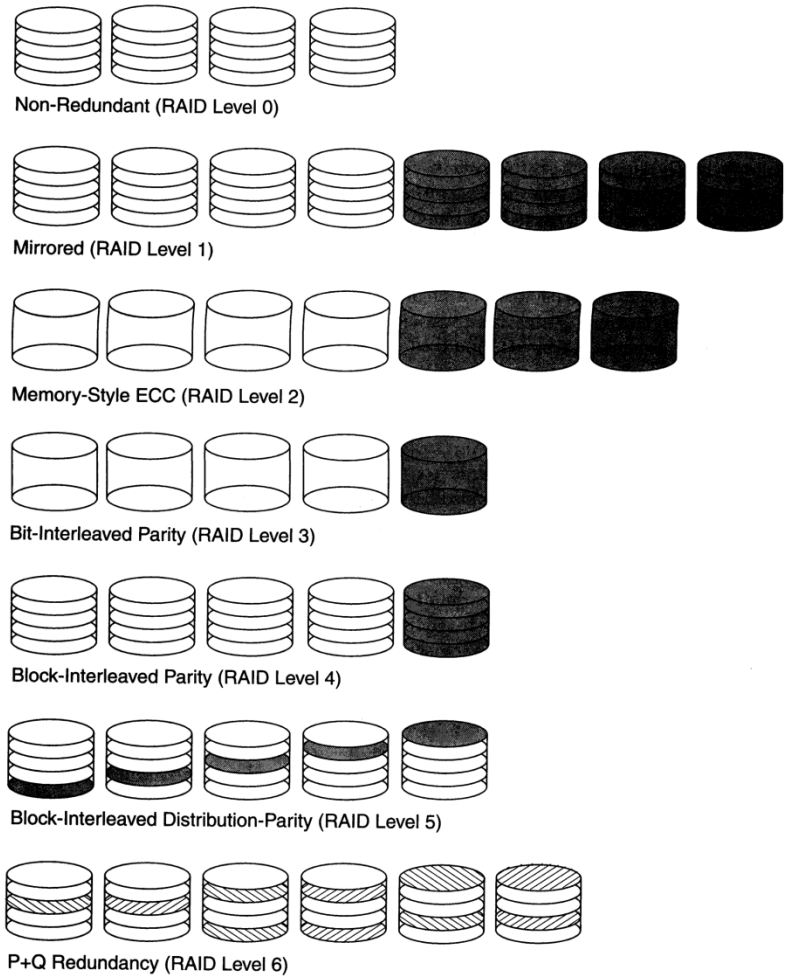
Use of RAID Technology (contd.)

- Different raid organizations are being used under different situations
  - Raid level 1 (mirrored disks) is the easiest for rebuild of a disk from other disks
    - It is used for critical applications like logs
  - Raid level 2 uses memory-style redundancy by using Hamming codes, which contain parity bits for distinct overlapping subsets of components.
    - Level 2 includes both error detection and correction.
  - Raid level 3 (single parity disks relying on the disk controller to figure out which disk has failed) and level 5 (block-level data striping) are preferred for Large volume storage, with level 3 giving higher transfer rates.

- Most popular uses of the RAID technology currently are:
  - Level 0 (with striping), Level 1 (with mirroring) and Level 5 with an extra drive for parity.

- Design Decisions for RAID include:
  - Level of RAID, number of disks, choice of parity schemes, and grouping of disks for block-level striping.

Use of RAID Technology (contd.)



Trends in Disk Technology

TABLE 13.3 TRENDS IN DISK TECHNOLOGY

	1993 PARAMETER VALUES*	HISTORICAL RATE OF IMPROVEMENT PER YEAR (%)*	CURRENT (2003) VALUES**
Areal density	50–150 Mbits/sq. inch	27	36 Gbits/sq. inch
Linear density	40,000–60,000 bits/inch	13	570 Kbits/inch
Inter-track density	1500–3000 tracks/inch	10	64,000 tracks/inch
Capacity (3.5" form factor)	100–2000 MB	27	146 GB
Transfer rate	3–4 MB/s	22	43–78 MB/sec
Seek time	7–20 ms	8	3.5–6 msec

**Source:* From Chen, Lee, Gibson, Katz, and Patterson (1994), *ACM Computing Surveys*, Vol. 26, No. 2 (June 1994). Reprinted by permission.

***Source:* IBM Ultrastar 36XP and 18ZX hard disk drives.

Storage Area Networks

- The demand for higher storage has risen considerably in recent times.
- Organizations have a need to move from a static fixed data center oriented operation to a more flexible and dynamic infrastructure for information processing.
- Thus they are moving to a concept of Storage Area Networks (SANs).
  - In a SAN, online storage peripherals are configured as nodes on a high-speed network and can be attached and detached from servers in a very flexible manner.
- This allows storage systems to be placed at longer distances from the servers and provide different performance and connectivity options.

Storage Area Networks (contd.)

- Advantages of SANs are:
  - Flexible many-to-many connectivity among servers and storage devices using fiber channel hubs and switches.
  - Up to 10km separation between a server and a storage system using appropriate fiber optic cables.
  - Better isolation capabilities allowing non-disruptive addition of new peripherals and servers.

- SANs face the problem of combining storage options from multiple vendors and dealing with evolving standards of storage management software and hardware.

Cloud Storage

- **Cloud storage** architecture offers an Internet based data model where an external *hosting* company is responsible for keeping the data.
- Cloud data is accessed typically through a web-based **API.**

- **Advantages:** unlimited ‘virtual’ capacities, low operating cost, reliable service, transparent technology, worry-free business model.

- **Concerns:** security, vulnerability, intrusion, hacking, espionage.

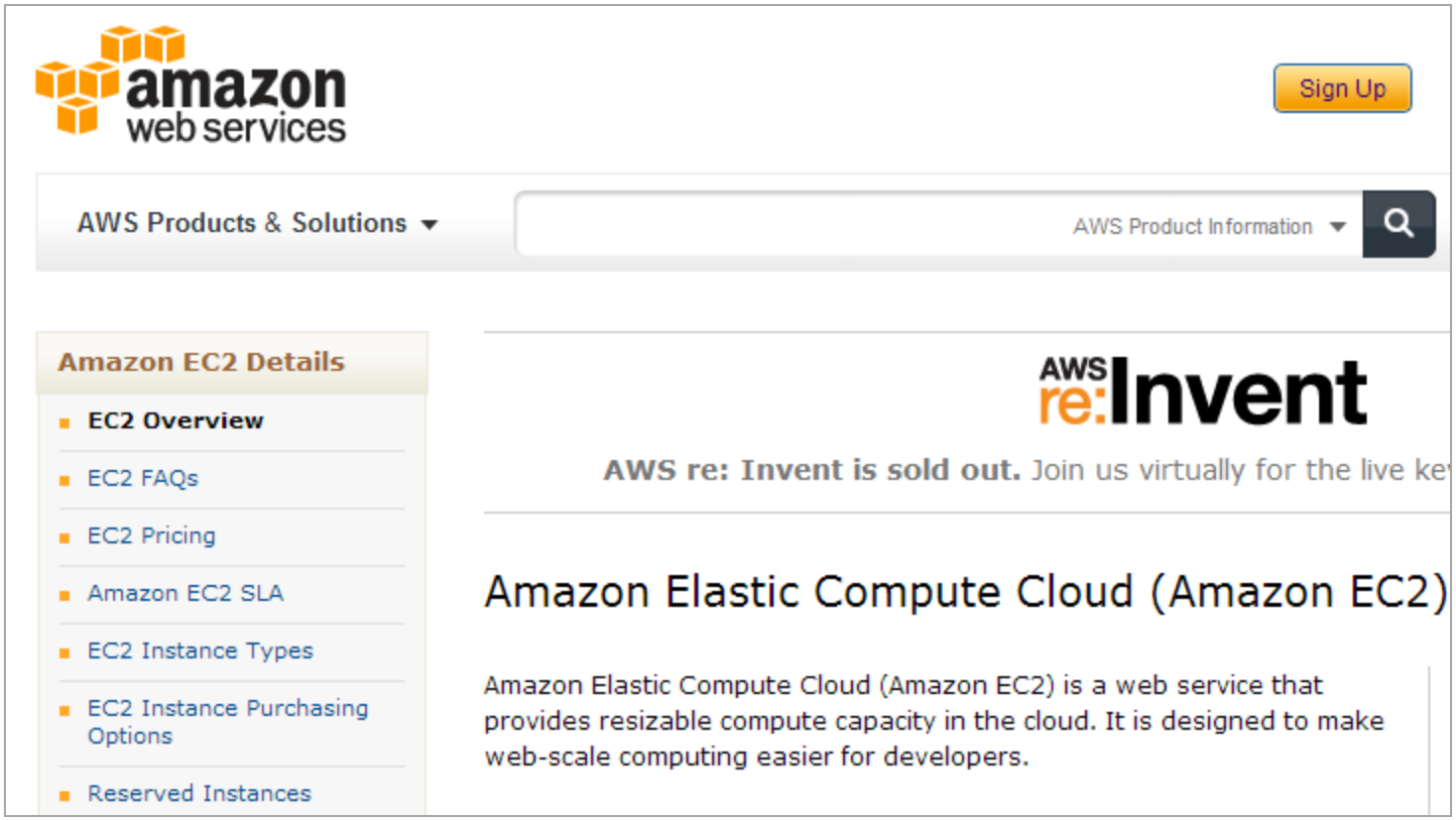
Cloud Storage

Free service for personal use

Cloud Storage

http://aws.amazon.com/ec2/

Paid service for enterprise computing



Cloud Storage

Paid storage options (annual price)

	SkyDrive	Apple iCloud	Google Drive	Dropbox
Free storage	7 GB	5 GB	5 GB	2 GB
Add 20 GB	$10	$40		
Add 50 GB	$25	$100		
Add 100 GB	$50		$60	$99

Reference
http://windows.microsoft.com/en-US/skydrive/compare

Summary

- Disk Storage Devices
- Files of Records
- Operations on Files
- Unordered Files
- Ordered Files
- Hashed Files
  - Dynamic and Extendible Hashing Techniques
- RAID Technology

Additional Reading

- P. Larson**, Dynamic hash tables** (Pdf) Communications of the ACM Volume 31, Issue 4 (April 1988) ISSN:0001-0782

- Litwin, Witold (1980), "**Linear hashing: A new tool for file and table addressing**" (PDF), *Proc. 6th Conference on Very Large Databases*: 212–223**.**

Appendix. Java Random Files

- *Random access files* permit non-sequential, or random, access to a file's contents.

- To access a file randomly, you open the file, seek a particular location, and read from or write to that file.

- Some code fragments:

```
File file = new File("c:\\temp\\myrandomdata.data");
RandomAccessFile randomfile = new RandomAccessFile(file, "rw");

int position = (i) * REC_SIZE;
randomfile.seek(position);

randomfile.writeBytes(stringData)
randomfile.read(bytebuffer);
```

Reference: http://docs.oracle.com/javase/tutorial/essential/io/rafs.html

Appendix. Example Java Random Files 1of 3

```
public class Drive {

  // ******************************************************************
  // Goal: Create a random access file, populate it, retrieve data. Records
  // are simple strings of length REC_SIZE
  // ******************************************************************
  final static int REC_SIZE = 20;
  static int[] keySet = { 1, 7, 3, 8, 12, 4, 11, 2, 10, 13, 14, 9 };
  final static int INITIAL_FILE_SIZE = keySet.length;

  public static void main(String[] args) {

    StringBuffer strbuffer = new StringBuffer(REC_SIZE);
    byte[] bytebuffer = new byte[REC_SIZE];

    try {
      // define a handler to reach the random file
      File file = new File("c:\\temp\\myrandomdata.data");
      RandomAccessFile randomfile = new RandomAccessFile(file, "rw");

      // preallocate  dummy entries in the file
      for (int i = 0; i < INITIAL_FILE_SIZE + 10; i++) {
        strbuffer = new StringBuffer();
        strbuffer.setLength(REC_SIZE);
        randomfile.writeBytes(strbuffer.toString().replace("\0", "-"));
      }
```

Appendix. Example Java Random Files 2 of 3

```
// Allocate (re-write) a number of test entries in the file
// each entry consists of Key(5char) data(rest)

for (int i = 0; i < INITIAL_FILE_SIZE; i++) {
      //get the corresponding starting byte position
      int position = (i) * REC_SIZE;
      randomfile.seek(position);

      // prepare the text data to be stored in each record
      String key = String.format("%05d", keySet[i] );
      String data = String.format("%15s", "[Data-row-" + i );

      strbuffer = new StringBuffer(key + data);
      strbuffer.setCharAt(REC_SIZE-1, ']');
      strbuffer.setLength(REC_SIZE);
      // write record on disk
      randomfile.writeBytes(strbuffer.toString().replace("\0", " "));

      System.out.printf("\nAllocating>>%2d:%2d:%s", i,
                     strbuffer.length(), strbuffer);

      // create hashing directory,
      // provide record key & record location

}

System.out.println("\nDone Allocating\n");
randomfile.close();
```

Appendix. Example Java Random Files 3 of 3

```
      // ////////////////////////////////////////////////////
      // Re-Open random access file, read some of its data
      // ////////////////////////////////////////////////////

      randomfile = new RandomAccessFile(file, "rw");
      for (int i = 0; i < INITIAL_FILE_SIZE + 10; i += 3) {
        int position = (i) * REC_SIZE;
        randomfile.seek(position);
        randomfile.read(bytebuffer);
        String strRecord = new String(bytebuffer);
        System.out.printf("\nReading %2d:%2d:%s", i,
            strRecord.length(), strRecord);
      }

    } catch (FileNotFoundException e) {
      e.printStackTrace();
    } catch (IOException e) {
      e.printStackTrace();
    }

  }

}
```

Appendix. Example Java Random Files 1 of 1

CONSOLE OUTPUT

```
Allocating>> 0:20:00001    [Data-row-]
Allocating>> 1:20:00007    [Data-row-]
Allocating>> 2:20:00003    [Data-row-]
Allocating>> 3:20:00008    [Data-row-]
Allocating>> 4:20:00012    [Data-row-]
Allocating>> 5:20:00004    [Data-row-]
Allocating>> 6:20:00011    [Data-row-]
Allocating>> 7:20:00002    [Data-row-]
Allocating>> 8:20:00010    [Data-row-]
Allocating>> 9:20:00013    [Data-row-]
Allocating>>10:20:00014   [Data-row-1]
Allocating>>11:20:00009   [Data-row-1]
Done Allocating


Reading  0:20:00001    [Data-row-]
Reading  3:20:00008    [Data-row-]
Reading  6:20:00011    [Data-row-]
Reading  9:20:00013    [Data-row-]
Reading 12:20:--------------------
Reading 15:20:--------------------
Reading 18:20:--------------------
Reading 21:20:--------------------
```